

Day 13 – Cyber Security & Ethical Hacking

DVWA (Damn Vulnerable Web Application): Learning How Poor Coding Creates Real Attacks

♦ Core Insight

- Most real-world cyber attacks **do not happen because of advanced hacking**
- They happen because of:
 - Poor coding practices
 - Missing security controls
 - Lack of developer awareness
- DVWA simulates these mistakes in a **safe, legal environment**

♦ Definition: What is DVWA?

- **DVWA (Damn Vulnerable Web Application)** is an intentionally insecure web application
- Designed for:
 - Security training
 - Vulnerability understanding
 - Ethical hacking education
- It contains **common real-world vulnerabilities**, configurable by difficulty level

♦ Why DVWA Is Important for Cyber Security Professionals

- It shows **how vulnerabilities are born**, not just how they are exploited
 - Helps understand:
 - Root cause of bugs
 - Impact on business systems
 - How attackers think
 - Bridges the gap between:
 - Developer mistakes
 - Attacker abuse
 - Defender mitigation
-

♦ Common Vulnerabilities Demonstrated in DVWA

- **SQL Injection**

- Caused by unsanitized user input
- Leads to unauthorized database access

- **Cross-Site Scripting (XSS)**

- Caused by improper output encoding
- Leads to session hijacking

- **Command Injection**

- Caused by trusting user input in system calls

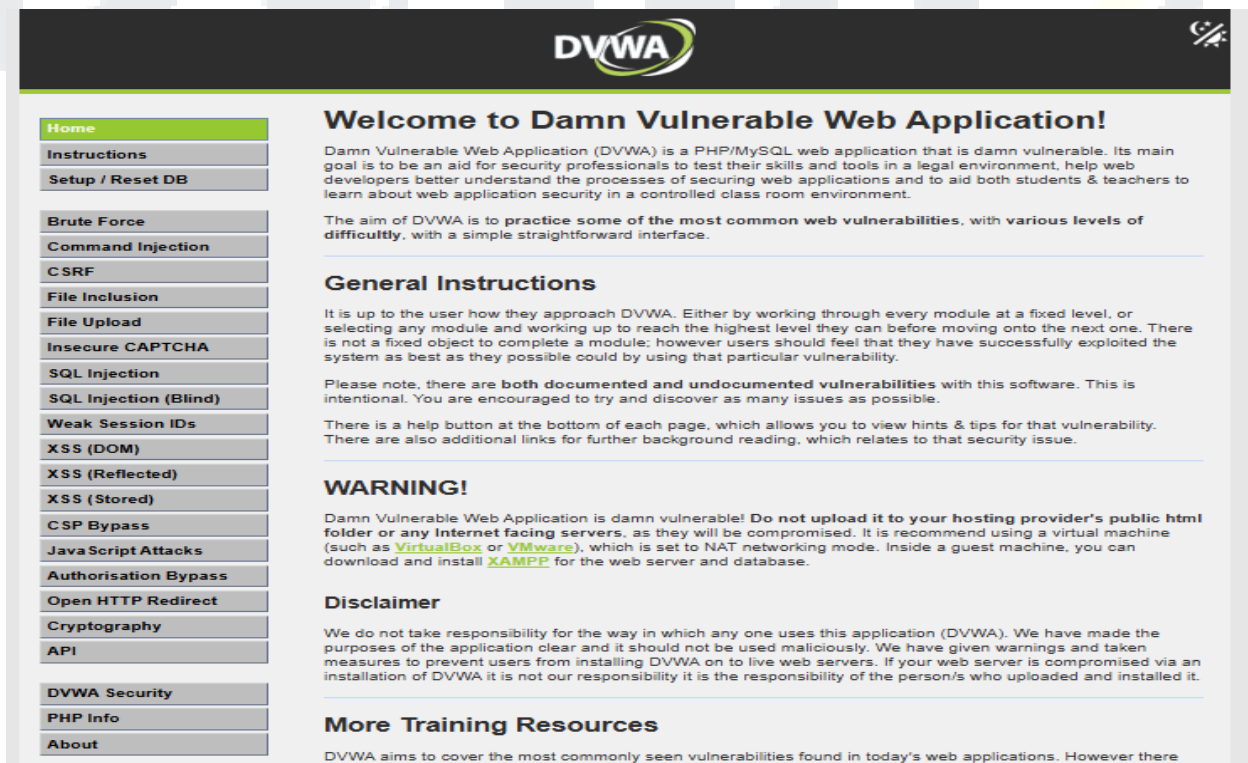
- **File Upload Issues**

- Caused by missing file validation

- **CSRF**

- Caused by lack of request verification

Note : These are not “DVWA problems” — they are **production problems**.



DVWA

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are both documented and undocumented vulnerabilities with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting provider's public html folder or any Internet facing servers, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

localhost/DVWA/vulnerabilities/brute/#

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass

Vulnerability: Brute Force

Login

Username:
Password:

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
Pinging 127.0.0.1 with 32 bytes of data:  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
  
Ping statistics for 127.0.0.1:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
    Approximate round trip times in milli-seconds:  
        Minimum = 0ms, Maximum = 0ms, Average = 0ms  
  
Pinging 8.8.8.8 with 32 bytes of data:  
Reply from 8.8.8.8: bytes=32 time=12ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=12ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=12ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=14ms TTL=118  
  
Ping statistics for 8.8.8.8:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
    Approximate round trip times in milli-seconds:  
        Minimum = 12ms, Maximum = 14ms, Average = 12ms
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

♦ Real-World Example (Enterprise Mapping)

- A company login page:
 - Accepts user input
 - Directly builds database queries
- Attacker manipulates input
- Result:
 - Authentication bypass
 - Data leakage
 - Regulatory violations

DVWA demonstrates **the same logic flaw**, safely.

♦ Developer vs Attacker Perspective

- Developer assumption:
 - “User will enter valid data”
- Attacker reality:
 - “User input is my weapon”
- DVWA exposes this **trust mismatch**
- Vulnerabilities are **design failures**, not tool failures
- Security must start at:
 - Code level
 - Architecture level
- DVWA teaches **why bugs exist**, not just how to find them
- Professionals use DVWA to:
 - Train mindset
 - Understand impact
 - Improve secure coding reviews

Master Conclusion

- Attacks succeed because **applications trust users**
- DVWA exposes this trust clearly
- Understanding vulnerable code is the foundation of:
 - Penetration testing
 - Secure development
 - Defensive security
- **Great hackers learn from bad code**



DVWA